

Análise Automatizada de Texto

Parte II. Oficina: do codebook à validação, aplicado ao seu projeto
SICSS-Brazil 2026 • FGV, Rio de Janeiro

Anderson Henrique

andersonheri@gmail.com

CEM/USP • INCT QualiGov • Pesquisador Convidado IPEA

Summer Institute in Computational Social Science, Brazil

29 de julho de 2026

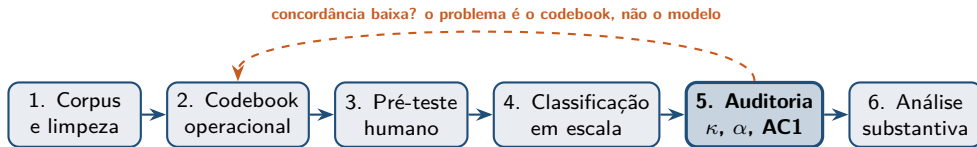
A tarde é uma oficina, não uma aula

Horário	Bloco	O que você leva
13h35 a 13h55	Exposição, 10 slides	O pipeline mínimo e as métricas
13h55 a 14h30	Trabalho individual	Codebook do seu projeto, testado em duas rodadas
14h30 a 14h50	Apresentações	Crítica dos colegas ao seu desenho
14h50 a 15h00	Síntese e fechamento	Roteiro das próximas duas semanas

Combinados

Os 20 minutos de exposição são o mínimo para a oficina fazer sentido. Todo o resto está no **apêndice** destes slides e no repositório. Nenhuma chave de API é necessária hoje.

O pipeline em seis etapas



O que a automação muda: apenas a etapa 4. As demais são idênticas à análise de conteúdo categorial clássica.

O que vocês farão hoje: as etapas 2 e 3, que são as que decidem se todo o resto se sustenta.

O que um codebook operacional precisa ter

- 1 **Nome curto e estável**, que vira nome de variável.
- 2 **Definição** em uma ou duas frases, sem circularidade.
- 3 **Critério de inclusão**: o que basta para atribuir.
- 4 **Critério de exclusão**: o que parece, mas não é.
- 5 **Dois exemplos-limite** do próprio corpus.
- 6 **Categoria residual** explícita.

Teste em trinta segundos

Codifique cinco documentos com o codebook. Depois, sem consultar o que você respondeu, codifique os mesmos cinco de novo.

Se as suas duas rodadas divergirem, o problema **nunca** é a memória. É a definição.

É exatamente esse teste que vocês farão daqui a vinte minutos, em duas rodadas cegas entre si.

Cinco erros que aparecem no pré-teste

Erro	Sintoma	Correção
Definição circular, como “discurso punitivista é o que é punitivista”	κ baixo e desacordo disperso	Substituir por marcadores observáveis
Categorias não exaustivas	Muitos casos não classificados	Criar categoria residual explícita
Categorias não exclusivas	Codificadores escolhem opções diferentes, ambas defensáveis	Hierarquizar, ou permitir múltiplos rótulos e mudar a métrica
Categoria muito abstrata	Concordância cai só nos casos-limite	Decompor em subcategorias observáveis
Prevalência abaixo de 5%	κ colapsa pelo paradoxo	Sobreamostrar no teste e reportar AC1

Guarde este slide para a atividade. Ele é a sua lista de diagnóstico quando a dupla-codificação der desacordo.

Como isso vira código

```
library(acR)

codebook <- ac_qual_codebook(
  name          = "posicao_seguranca",
  instructions = paste(
    "Classifique a posicao predominante do pronunciamento sobre",
    "seguranca publica. Use APENAS o conteudo manifesto do texto.",
    "Se o texto nao tratar do tema, use 'nao_aplicavel'."),
  categories = list(
    punitivista  = list(definition = "Defende aumento de pena, reducao da
                                     maioridade penal ou uso da forca."),
    preventivo    = list(definition = "Enfatiza causas sociais: educacao,
                                     emprego, assistencia."),
    garantista   = list(definition = "Enfatiza direitos do acusado e
                                     controle da policia."),
    nao_aplicavel = list(definition = "O texto nao trata do tema.)),
  mode = "manual")  # as categorias sao SUAS, nao do modelo
```

A estrutura do objeto é a mesma do codebook em papel.

Classificação com LLM, e o que ela devolve

```
chat_obj <- ellmer::chat_anthropic(model = "claude-sonnet-4-20250514")

resultado <- ac_qual_code(corpus      = corpus_limpo,
                          codebook    = codebook,
                          chat         = chat_obj,
                          k_consistency = 3L,    # 3 passagens, voto majoritario
                          confidence   = "total")

# NO WORKSHOP: carregue o resultado salvo, nao rode o bloco acima
resultado <- readRDS("data/resultado_llm_precomputado.rds")
```

```
<ac_qual_result> 300 docs | modelo: claude-sonnet-4-20250514
  categoria      n      conf_media k_unanime
punitivista    118      0.91      0.86
preventivo      84      0.83      0.71
garantista      27      0.68      0.48  <- alta hesitacao
nao_aplicavel   71      0.95      0.94
```

A chave de API vai no .Renviron, nunca no script: `usethis::edit_r_environ()`.

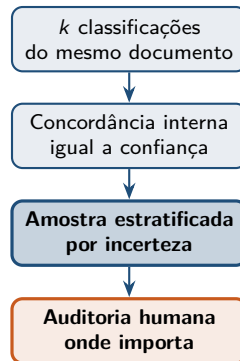
A hesitação do modelo é informação, não defeito

Executar o mesmo *prompt* k vezes e comparar:

- **3 de 3 iguais:** documento fácil.
- **2 de 3:** zona de fronteira. É aqui que o humano deve olhar.
- **Três respostas distintas:** ou o documento é ambíguo, ou o codebook não cobre o caso.

Isso converte o não determinismo, que é uma limitação, em instrumento de amostragem.

Cuidado: confiança do modelo não é probabilidade de acerto. É consistência. Modelos podem ser consistentemente errados.



Amostragem e métricas de confiabilidade

```
# Amostra por incerteza, planilha sem rotulo, revisao humana
amostra <- ac_qual_sample(resultado, n = 150, strategy = "uncertainty")
ac_qual_export_for_review(amostra, "outputs/revisao.xlsx", corpus_limpo)
humano <- ac_qual_import_human("outputs/revisao_preenchida.xlsx")

# Kappa/alpha vem de ac_qual_irr(); AC1/F1 exigem ac_qual_reliability()
irr      <- ac_qual_irr(gold = humano, predicted = resultado,
                       id_col = "doc_id", cat_col = "categoria")
confiab <- ac_qual_reliability(llm = resultado, human = humano)
```

```
n = 150 | Percent agreement 0.833 | Cohen's kappa 0.760 [0.66, 0.86]
ac_qual_reliability(bootstrap, n=1000): alpha 0.761 [0.67, 0.84]
  Gwet's AC1 0.783 [0.71, 0.86] | F1 macro 0.786 [0.70, 0.86]
```

Por que o número agregado engana

categoria	n	precisao	revocacao	F1	kappa_cat	
punitivista	61	0.918	0.862	0.889	0.809	
preventivo	43	0.767	0.805	0.786	0.702	
garantista	17	0.471	0.667	0.552	0.505	<- fraco
nao_aplicavel	29	0.966	0.875	0.918	0.897	

O que este quadro autoriza, e o que ele proíbe

$\kappa = 0,76$ é publicável. Mas “garantista” tem precisão de 0,47: quase metade das vezes que o modelo aponta essa categoria, ele erra.

Conclusão honesta: o pipeline sustenta análise sobre punitivismo e prevenção. Não sustenta afirmação alguma sobre a trajetória do discurso garantista. Reportar apenas o κ agregado esconderia isso.

Reporte sempre por categoria.

A matriz de confusão é o diagnóstico do codebook

Leia a linha “garantista”: dos 17 casos, o modelo acertou 8 e distribuiu os demais entre punitivista e preventivo.

Isso não é ruído aleatório. É uma **fronteira conceitual mal definida** entre defender direitos do acusado e defender prevenção social.

A correção não está no modelo. Está na frase do codebook.

Saída ilustrativa, gerada para fins didáticos.

Matriz de confusão (n = 150)

Classificação do LLM	punitivista	56	3	1	1
	preventivo	5	33	3	2
	garantista	4	4	8	1
	não aplic.	0	1	0	28
		punitivista	preventivo	garantista	não aplic.
		Codificação humana (gold standard)			

O mínimo inegociável, em cinco linhas

- 1 Rotule uma **amostra aleatória por humanos**, mesmo pequena, entre 100 e 300 documentos. Ela é o seu *gold standard*.
- 2 Reporte κ , α e AC1 **por categoria**, com a matriz de confusão.
- 3 Rode um *baseline* simples antes do modelo complexo, e mostre os dois.
- 4 Trate rótulos de máquina como **medida com erro**, e não como dado observado.
- 5 Deposite *prompt*, versão do modelo, *outputs* brutos e `sessionInfo()` em repositório público.

O detalhamento de cada item está no apêndice, ao final deste arquivo.

Atividade: 35 minutos, individual, no seu próprio projeto

Objetivo

Sair daqui com o esqueleto validável de uma medida de texto para **a sua própria pesquisa**, já submetido a um teste de confiabilidade.

Tempo	Etapa	Produto
5 min	Defina a pergunta e a unidade de registro do seu projeto	Pergunta e unidade de registro definidas
15 min	Escrever o codebook, de 2 a 4 categorias	Definição, inclusão, exclusão e 2 exemplos-limite por categoria
10 min	Dupla-codificação interna: aplique o codebook duas vezes	5 documentos do seu corpus, codificados em duas rodadas cegas entre si
5 min	Calcular concordância e diagnosticar	A categoria com maior desacordo, e a reescrita proposta

Materiais: `scripts/06_atividade.R` (script-modelo do acR) e `data/corpus_atividade.csv` (exemplo, se ainda não tiver corpus próprio). Quem preferir papel, use papel: o produto é o raciocínio, não o arquivo.

Como conduzir a dupla-codificação interna

Regras

- 1 Escolha 5 documentos do seu corpus (de preferência, casos difíceis) e codifique-os com o codebook: rodada 1.
- 2 Depois de alguns minutos, sem consultar a rodada 1, codifique os mesmos 5 documentos de novo: rodada 2.
- 3 Só então compare as duas rodadas.

O cálculo, sem software

Concordância bruta = acordos entre as duas rodadas dividido por 5.

Com 5 documentos, nenhuma métrica corrigida pelo acaso é confiável. O objetivo aqui não é o número, é **localizar o desacordo**.

A pergunta que interessa

Para cada discordância: você leu o documento de forma diferente na segunda rodada, ou aplicou uma regra diferente? O primeiro caso é problema de **unidade de registro**. O segundo, de **definição**.

Apresentações: 3 minutos por pessoa

Roteiro fixo. Cronometrado.

- 1 A pergunta de pesquisa e a unidade de registro escolhida. (30 s)
- 2 Uma categoria do codebook, lida em voz alta. (30 s)
- 3 A categoria com maior desacordo entre as duas rodadas, e o diagnóstico: circular, não exaustiva, não exclusiva, abstrata demais ou rara demais? (60 s)
- 4 A reescrita proposta para essa definição. (30 s)
- 5 Qual método você usaria, entre dicionário, supervisionado e LLM, e por quê. (30 s)

Papel da plateia

Quem apresenta recebe **uma** pergunta de outro participante, escolhido pelo facilitador. Pergunta, não sugestão. A melhor costuma ser: “que documento faria essa categoria falhar?”

O critério de sucesso desta atividade

Não é ter um codebook perfeito

É ter descoberto onde ele quebra **antes** de gastar dinheiro, tempo e reputação.

Quem apresenta um desacordo bem diagnosticado
aprendeu mais do que quem apresenta
concordância perfeita em cinco documentos fáceis.

Se o seu codebook sobreviveu intacto ao teste, desconfie: ou as categorias são triviais, ou os cinco documentos eram fáceis demais.

As próximas duas semanas, no seu projeto

- 1 Semana 1.** Monte o corpus e rode apenas descritivas. Leia 30 documentos a olho. Você vai revisar o codebook depois disso, e todo mundo revisa.
- 2 Semana 1.** Codifique 50 documentos manualmente. Meça concordância com um colega. Reescreva o codebook.
- 3 Semana 2.** Rode o *baseline* de dicionário e o LLM em 300 documentos. Compare com os seus 50 rótulos humanos.
- 4 Semana 2.** Só então escale. E deposite tudo em repositório público desde o primeiro dia, não no fim.

Fico à disposição

Dúvidas sobre o acR, *issues* e sugestões: github.com/andersonheri/acR/issues • andersonheri@gmail.com

Síntese das duas aulas

- 1 A escala não relaxa nenhuma exigência metodológica. Ela transfere o ônus da codificação para a auditoria.
- 2 Identifique a tarefa antes da ferramenta, e use *baseline* sempre.
- 3 Pré-processamento é decisão teórica e precisa ser reportado.
- 4 Concordância se reporta por categoria, com mais de uma métrica e com a matriz de confusão.
- 5 Rótulos gerados por máquina carregam erro sistemático.
- 6 Transparência é o que separa análise automatizada de texto de geração automatizada de números.

“*Validate, validate, validate*”, Grimmer e Stewart (2013). Treze anos depois, com LLMs, o conselho ficou mais urgente.

Referências (1 de 2)

Base metodológica

GRIMMER, Justin; ROBERTS, Margaret E.; STEWART, Brandon M. **Text as Data: a new framework for machine learning and the social sciences**. Princeton: Princeton University Press, 2022.

BARBERÁ, Pablo et al. Automated text classification of news articles: a practical guide. *Political Analysis*, v. 29, n. 1, p. 19–42, 2021.

KRIPPENDORFF, Klaus. **Content Analysis: an introduction to its methodology**. 4. ed. Thousand Oaks: Sage, 2018.

SAMPAIO, Rafael Cardoso; LYCARIÃO, Diógenes. **Análise de conteúdo categorial: manual de aplicação**. Brasília: Enap, 2021.

Referências (2 de 2)

LLMs, confiabilidade e software

GILARDI, Fabrizio; ALIZADEH, Meysam; KUBLI, Maël. ChatGPT outperforms crowd workers for text-annotation tasks. *PNAS*, v. 120, n. 30, 2023.

WANG, Xuezhi et al. Self-consistency improves chain of thought reasoning in language models. *EMNLP*, 2023.

GWET, Kilem L. **Handbook of Inter-Rater Reliability**. 4. ed. Gaithersburg: Advanced Analytics, 2014.

BENOIT, Kenneth et al. quantda: an R package for the quantitative analysis of textual data. *Journal of Open Source Software*, v. 3, n. 30, 2018.

HENRIQUE, Anderson. **acR**: análise de conteúdo em R. Pacote R versão 0.3.2, 2026. ahenriquecp.com/acR
[VERIFICAR] Literatura sobre inferência com rótulos imperfeitos, publicada entre 2023 e 2026.

Obrigado!

Anderson Henrique

CEM/USP • INCT QualiGov • IPEA

andersonheri@gmail.com

Materiais, dados e scripts comentados:

`github.com/andersonheri/sicss-brazil-2026-text`

Pacote acR: `ahenriquecp.com/acR`

Como citar: HENRIQUE, Anderson. *Análise Automatizada de Texto: oficina de codebook, validação e reprodutibilidade em R*. 2026. Apresentação em slides: material didático não publicado, elaborado para o Summer Institute in Computational Social Science, Brazil (SICSS-Brazil 2026), Fundação Getulio Vargas, Rio de Janeiro. CEM-USP / INCT QualiGov.

Apêndice

Material de consulta

Estes slides não fazem parte da exposição de 20 minutos. Estão aqui para as dúvidas da oficina e para consulta posterior.

A2. A escada de *baselines*

Um resultado que a literatura repete

Modelos complexos costumam superar modelos simples por margens pequenas em classificação de texto político. Sem *baseline*, você não sabe se o ganho justificou o custo, a opacidade e a perda de replicabilidade.

- 1 Classe majoritária, que é o piso absoluto.
- 2 Dicionário construído a partir do seu codebook.
- 3 Regressão logística ou Naive Bayes sobre a matriz documento-termo.
- 4 LLM *zero-shot*.
- 5 LLM *few-shot* com exemplos do seu corpus.

Reporte todos. Uma tabela com cinco linhas e a coluna F_1 por classe é uma das seções de métodos mais convincentes que você pode escrever.

A3. Código do *baseline* supervisionado

```
library(quantda); library(quantda.textmodels)
set.seed(1234)

# Particao ESTRATIFICADA pela classe (importante com classes raras)
idx <- caret::createDataPartition(rotulos$categoria, p = .8, list = FALSE)

mod_nb <- textmodel_nb(dfm_treino[idx, ], rotulos$categoria[idx])
pred <- predict(mod_nb, newdata = dfm_teste)

caret::confusionMatrix(pred, rotulos$categoria[-idx], mode = "prec_recall")
```

```
Accuracy : 0.7533      (No Information Rate : 0.4067)
          Precision Recall F1
Class: punitivista    0.8033 0.8033 0.8033
Class: preventivo      0.7000 0.6512 0.6747
Class: garantista     0.4444 0.2353 0.3077 <- classe rara falha
Class: nao_aplicavel  0.8710 0.9310 0.9000
```

A4. Três armadilhas no código anterior

1 **Partição não estratificada.**

Com classes raras, uma partição aleatória simples pode deixar a categoria “garantista” quase ausente do treino.

2 **Vazamento de informação.**

Construir a matriz documento-termo antes de separar treino e teste faz o vocabulário do teste influenciar o treino.

3 **Acurácia global como métrica.**

Se 70% do corpus é uma só classe, um classificador trivial atinge 70% de acurácia sem aprender nada. Compare sempre com a taxa de não informação.

A5. Categorias definidas por você ou pelo modelo?

Categorias *a priori*

- Você define a partir da teoria.
- Corresponde à análise de conteúdo dedutiva.
- Recomendado quando há literatura consolidada.

Categorias emergentes

- Úteis na fase exploratória.
- Mas, se o modelo propõe as categorias e também as aplica, não há validação independente.

O problema epistemológico

Deixar o LLM inferir as categorias e depois usá-lo para codificar cria circularidade: a concordância medida é a consistência interna do modelo, não a validade da categoria. Nesse caso, a validação humana passa a ser obrigatória.

A6. Amostragem por incerteza ou aleatoriamente?

Por incerteza

- Concentra o esforço humano onde o modelo hesitou.
- Diagnostica o codebook com eficiência.
- **Superestima** o desacordo.

Aleatória

- Estima a acurácia populacional sem vies.
- Permite corrigir estimativas a jusante.
- Desperdiça esforço nos casos fáceis.

Faça as duas, e reporte separadamente

A amostra aleatória é o seu *gold standard* para inferência. A amostra por incerteza é a sua ferramenta de diagnóstico. Confundir as duas produz uma taxa de erro que não descreve nem uma coisa nem outra.

A8. Riscos que exigem tratamento explícito no artigo

- 1 **Não determinismo.** O mesmo *prompt*, em datas diferentes, pode gerar rótulos diferentes. Fixe versão, temperatura e *seed*, e reporte a data de execução.
- 2 **Opacidade e contaminação.** Você não sabe o que há no treino. Contaminação com o próprio corpus é possível e indetectável.
- 3 **Viés de posição e de rótulo.** Modelos favorecem categorias listadas primeiro. Teste embaralhando a ordem.
- 4 **Deriva de versão.** O provedor atualiza o modelo e seu resultado deixa de replicar. Salve os *outputs* brutos.
- 5 **Custo financeiro e energético.** Reporte o volume de chamadas.

Regra que proponho

O *output* bruto do modelo é **dado primário** e deve ser depositado junto com o corpus. Sem isso, o trabalho é irreplicável por construção.

A9. Relatório reprodutível

```
# Relatório metodológico automático: codebook, parametros,  
# metricas, amostra auditada e sessionInfo  
ac_qual_report(resultado, codebook = codebook, reliability = confiab,  
               format = "html", path = "outputs/relatorio_metodologico.html")  
  
ac_export(resultado, path = "outputs/classificacao.csv")  
ac_export(irr,      path = "outputs/metricas.tex")    # direto para o artigo  
  
writeLines(capture.output(sessionInfo()), "outputs/sessionInfo.txt")
```

O que vai para o repositório público

Corpus, quando a licença permitir; codebook; *prompts* completos; *outputs* brutos do modelo; planilha de codificação humana anonimizada; scripts numerados; `sessionInfo()`; e `renv.lock`.

A10. *Checklist* de reporte

Desenho

- ☐ Unidade de amostragem, registro e contexto
- ☐ Critério de inclusão no corpus e período
- ☐ Codebook completo em anexo
- ☐ Todas as decisões de pré-processamento

Classificação

- ☐ *Baseline* e comparação
- ☐ Modelo, versão exata e data de execução
- ☐ *Prompt* completo, não parafraseado
- ☐ Parâmetros: temperatura, k e *seed*

Validação

- ☐ n e estratégia da amostra humana
- ☐ Número de codificadores e treinamento
- ☐ κ , α e AC1 por categoria
- ☐ Matriz de confusão
- ☐ F_1 por classe

Transparência

- ☐ Repositório público
- ☐ `sessionInfo()` e `renv.lock`
- ☐ Declaração de uso de IA
- ☐ Limitações por categoria